

Reviewer Pack — SAW Connective Growth of Gate DAG Lower-Bounds $\text{ACC}^0[2]$ for ...

Entry 988e93f4afe3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 02:53:55 UTC

SAW Connective Growth of Gate DAG Lower-Bounds $\text{ACC}^0[2]$ for MOD_3

Entry ID: 988e93f4afe3 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 02:53:55 UTC

1. Conjecture

Field A (mathematical branch): Statistical mechanics of self-avoiding walks — Hammersley’s connective constant $\mu(G) = \lim_L c_L(G)^{1/L}$ on finite graphs (Hammersley 1957; Madras-Slade 1993 ‘The Self-Avoiding Walk’; Duminil-Copin-Smirnov 2012 honeycomb constant $\sqrt{(2+\sqrt{2})}$; Grimmett-Li 2017 ‘Connective constants and height functions for Cayley graphs’). A combinatorial-physics functional based on the set-predicate ‘walk is self-avoiding’ — counts can be enumerated by DFS with a visited-set, an $O(L \cdot \deg^L)$ routine; no F_q polynomial extension exists since ‘no vertex repeats’ is a Boolean lattice predicate on power-set membership, not a polynomial identity, so the bridge is Aaronson-Wigderson algebrization-safe. The invariant is STRUCTURAL on the gate-incidence multigraph G_C (two $\text{ACC}^0[2]$ circuits computing the same Boolean function can have different connective growth), shielding from Razborov-Rudich natural proofs. Distinct from blacklisted ACC^0 attempts: Plünnecke doubling (sumset on top-Fourier supports, NATURAL_PROOFS-rejected), Sidon r^* (max representation in F_2^N on MOD-gate outputs), Chang spectrum (Fourier η -cardinality on wire labels), additive energy (L^2 on out-degree sequence), Gromov 4-point δ (max over distance differences), Forman-Ricci (Bochner edge-vertex sum), Mostar (edge distance asymmetry), Dilworth width (König matching on poset), SoS cone-gram stable rank (Schatten $1/\infty$ on Gram), Mean log-bias (KL on internal gate biases) — none uses SAW enumeration on the gate-incidence multigraph. An arXiv search ‘self-avoiding walk’ AND (‘ ACC^0 ’ OR ‘Williams algorithmic method’ OR ‘NEXP circuit lower bound’) returns 0 direct hits and <5 adjacent papers (all on SAWs on Cayley graphs in mathematical physics, never on ACC circuits).

Field B (complexity object): $\text{ACC}^0[2]$ circuit size $s(C)$ and depth $d(C)$ for the explicit function MOD_3 on n inputs ($q=3$ coprime to $m=2$), in the Williams 2011 / Williams 2014 ‘Nonuniform ACC circuit lower bounds’ / Murray-Williams 2018 algorithmic-method regime targeting $\text{NEXP} \not\subseteq \text{ACC}^0$ and explicit $\text{ACC}^0[2]$ lower bounds for MOD_3 . The canonical Razborov-Smolensky stepping stone: MOD_3 has no $o(\sqrt{n})$ -degree F_2 polynomial approximation, so any $\text{ACC}^0[2]$ biased toward MOD_3 must wire its DAG to accommodate this — predicting a structural lower bound on a non-spectral combinatorial invariant of the gate graph.

Statement:

Let C be an $\text{ACC}^0[2]$ circuit on n inputs with $s = |\text{gates}(C)|$, treated as an undirected multigraph G_C on gate vertices with an edge for every wire (parallel edges retained). Fix walk-length $L := \lfloor 2 \cdot \log_2(s+2) \rfloor$ and let $c_L(G_C)$ be the number of self-avoiding walks of length L starting from the output gate. Define the SAW connective growth $\rho(C) :=$

$\log_2(c_L(G_C)+1)/L$. The conjecture: for every $ACC^0[2]$ circuit C whose Boolean function f_C agrees with MOD_3 on at least $0.45 \cdot 2^n$ uniform inputs (bias $|\Pr[f_C=MOD_3] - 1/3| \geq 1/9$), the growth satisfies $\rho(C) \geq (1/8) \cdot \log_2(s) - 1$; equivalently, $c_L(G_C) \geq s^{\{L/8\}/2} L$. One counterexample (a biased circuit with low ρ) refutes it.

Rationale (proposer’s reasoning):

MOD_3 is Razborov-Smolensky-hard to F_2 -approximate by low-degree polynomials, so any $ACC^0[2]$ circuit that biases toward MOD_3 must repeatedly funnel many F_2 -linear gate outputs through interleaved AND/OR layers; this wiring creates the local branching and reconvergence that drive SAW count growth on G_C . SAW growth is a non-spectral, non-Fourier coarse-geometric obstruction that survives where ring-level invariants algebrize, and it is structural rather than functional, so it sidesteps Razborov-Rudich largeness while remaining sensitive to the underlying $ACC^0[2]$ resource. This puts the SAW connective constant in the same coarse niche as Forman-Ricci or Gromov hyperbolicity but with a fundamentally different (set-predicate, lattice-walk) computation.

Taxonomy category: $ACC_LB_via_WILLIAMS$ (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c65571d8fab13827

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 30 seeds generating $ACC^0[2]$ circuits on $n \in \{8, 10, 12, 14, 16\}$ with $s \in \{12, 16, 20, 24, 28, 32, 36, 40\}$, retain instances with $|\Pr[f_C=MOD_3] - 1/3| \geq 1/9$. Conjecture is SUPPORTED iff every retained instance satisfies $\rho(C) \geq (1/8) \cdot \log_2(s) - 1$, with ≥ 50 retained instances total; FALSIFIED if any retained instance violates.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.86	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.78	SAFE	SAFE
ALGEBRIZATION	SAFE	0.78	SAFE	SAFE
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 7 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - self-avoiding walk connective constant circuit complexity lower bound - $ACC^0 MOD_3$ gate graph combinatorial invariant Williams algorithmic method - self-avoiding walk enumeration Boolean circuit DAG Razborov Smolensky

Top relevant hits considered: - [http://arxiv.org/abs/1302.2796v2] Endless self-avoiding walks - [http://arxiv.org/abs/1307.6288v2] Self-avoiding quantum walks: realisations in subspaces and limit theorems - [http://arxiv.org/abs/hep-lat/9302003v1] New Lower Bounds on the Self-Avoiding-Walk Connective Constant - [http://arxiv.org/abs/1009.3029v1] Invariant Spectral Hashing of Image Saliency Graph - [http://arxiv.org/abs/2206.07869v1] Let Invariant Rationale Discovery Inspire Graph Contrastive Learning - [http://arxiv.org/abs/2510.02587v2] A combinatorial formula for Interpolation Macdonald polynomials - [http://arxiv.org/abs/1612.07234v5] Scaling limit of a self-avoiding walk interacting with spatial random permutations

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from collections import defaultdict

def generate_circuit(n, s):
    depth = 3
    circuit = []
    for _ in range(depth):
        layer = []
        for _ in range(s // depth):
            if _ % 2 == 0:
                gate_type = 'AND'
            else:
                gate_type = 'OR'
            layer.append(gate_type)
        circuit.extend(layer)
    return circuit

def simulate_circuit(circuit, n):
    stack = []
    for gate in circuit:
        if gate == 'NOT':
            a = stack.pop()
            stack.append(not a)
        elif gate == 'AND':
            b, a = stack.pop(), stack.pop()
            stack.append(a and b)
        elif gate == 'OR':
            b, a = stack.pop(), stack.pop()
            stack.append(a or b)
    return stack[0]

def estimate_bias(circuit, n):
    N = 2048
    count = 0
    for _ in range(N):
        result = simulate_circuit(circuit, n)
        if result % 3 == 1:
            count += 1
    bias = count / N
    return bias

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [8, 10, 12, 14, 16]
    s_values = [12, 16, 20, 24, 28, 32, 36, 40]
    L_max = 14
```

```

results = []
for n in n_values:
    for s in s_values:
        circuit = generate_circuit(n, s)
        bias = estimate_bias(circuit, n)
        if abs(bias - 1/3) >= 1/9:
            L = math.floor(2 * math.log2(s + 2))
            if L > L_max:
                continue
            c_L = 0
            visited = [False] * (s + 1)
            stack = [(s, [])]
            while stack:
                node, path = stack.pop()
                if len(path) == L and node == s:
                    c_L += 1
                    continue
                for i in range(s):
                    if not visited[i]:
                        visited[i] = True
                        stack.append((i, path + [i]))
                        visited[i] = False
            rho = math.log2(c_L + 1) / L
            results.append({
                "n": n,
                "s": s,
                "bias": bias,
                "rho": rho,
                "c_L": c_L,
                "L": L
            })

if not results:
    return {
        "metric_name": "rho",
        "metric_value": 0.0,
        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "No biased circuits found"
    }

rho_values = [result["rho"] for result in results]
bias_values = [result["bias"] for result in results]
n_tests = len(results)

if all(rho >= (1/8) * math.log2(s) - 1 for s, _, rho in results):
    return {
        "metric_name": "rho",
        "metric_value": sum(rho_values) / n_tests,
        "instances_tested": n_tests,
        "conjecture_holds": True,
        "counterexample": ""
    }
else:
    counterexample = f"Counterexample found with bias={max(bias_values)} and rho={min(rho_valu"
    return {

```

```

        "metric_name": "rho",
        "metric_value": sum(rho_values) / n_tests,
        "instances_tested": n_tests,
        "conjecture_holds": False,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i - 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    rho_values = [result["metric_value"] for result in results if "metric_value" in result]
    instances_tested = sum(result["instances_tested"] for result in results if "instances_tested" in result)
    support_fraction = sum(1 for result in results if result.get("conjecture_holds", False)) / len(results)

    if all(rho >= (1/8) * math.log2(s) - 1 for s, _, rho in results):
        print(f"RESULT: SUPPORTED mean={sum(rho_values)/instances_tested} std=0.0 support_fraction={support_fraction}")
    elif any(not result.get("conjecture_holds", False) for result in results):
        counterexample = next(result["counterexample"] for result in results if not result.get("conjecture_holds", False))
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result.get("conjecture_holds", False))
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c639def0.py", line 131, in <module>
    trial_result = run_trial(seed)
                   ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c639def0.py", line 66, in run_trial
    bias = estimate_bias(circuit, n)
           ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c639def0.py", line 49, in estimate_bias
    result = simulate_circuit(circuit, n)
             ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c639def0.py", line 38, in simulate_circuit
    b, a = stack.pop(), stack.pop()
           ~~~~~^~~~~~
IndexError: pop from empty list

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an `IndexError` (pop from empty stack) in `simulate_circuit` before any retained instances could be evaluated, so no data was produced to assess the pre-registered support or falsification condition. | next: Fix the circuit simulator's stack-handling bug (ensure each gate has sufficient operands on the stack, likely by validating circuit generation or adjusting evaluation order) and rerun the 30-seed sweep.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	154886
2	preregistration	claude_max	opus	0	0	5145
3	novelty	claude_max	opus	0	0	3702
4	novelty	claude_max	opus	0	0	7090
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18380
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13222
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11927
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14203
9	judge	claude_max	opus	0	0	4124

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 232680 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 4}

(full prompt+response transcripts available in `research/audit/988e93f4afe3.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/988e93f4afe3.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/988e93f4afe3.tar.gz` (if generated)